

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Bai, Shaojie, J. Zico Kolter, and Vladlen Koltun., arXiv preprint arXiv:1803.01271 (2018)

BOAZ 25기 분석 발제 - TCN
김지수 이선경

Table of Contents

- 01** Introduction
- 02** Background
- 03** Temporal Convolutional Networks (TCN)
- 04** Sequence Modeling Tasks
- 05** Experiments
- 06** Conclusion
- 07** Appendix

01 Introduction

Abstract

For most deep learning practitioners, sequence modeling is synonymous with recurrent networks. Yet recent results indicate that **convolutional architectures can outperform recurrent networks on tasks such as audio synthesis and machine translation**. Given a new sequence modeling task or dataset, which architecture should one use? We conduct a systematic evaluation of generic convolutional and recurrent architectures for sequence modeling. The models are evaluated across a broad range of standard tasks that are commonly used to benchmark recurrent networks. Our results indicate that a **simple convolutional architecture outperforms canonical recurrent networks such as LSTMs across a diverse range of tasks and datasets, while demonstrating longer effective memory**. We conclude that the common association between sequence modeling and recurrent networks should be reconsidered, and convolutional networks should be regarded as a natural starting point for sequence modeling tasks. To assist related work, we have made code available at <http://github.com/locuslab/TCN>.

1. Common belief in sequence modeling

2. Value of Convolutional architecture compared to RNN

01 Introduction

- In deep learning practice... → “**sequence = RNN(LSTM/GRU)**”
- But in recent studies...
 - **CNNs have reached SOTA** in areas such as audio synthesis, word-level LM, and translation etc.

Is this a domain-specific phenomenon or a principle that applies more broadly?

We address this question by conducting a systematic empirical evaluation of convolutional and recurrent architectures on a broad range of sequence modeling tasks. We specifically target a comprehensive set of tasks that have been repeatedly used to compare the effectiveness of different recurrent network architectures. These tasks include polyphonic music modeling, word- and character-level language modeling, as well as synthetic stress tests that had been deliberately designed and frequently used to benchmark RNNs. Our evaluation is thus set up to compare convolutional and recurrent approaches to sequence modeling on the recurrent networks’ “home turf”.

CNN vs RNN

- TCN for CNN representation
- LSTM and GRU(canonical architectures) for RNN

02 Background - 5 key points

Convolutional networks (LeCun et al., 1989) have been applied to sequences for decades (Sejnowski & Rosenberg, 1987; Hinton, 1989). They were used prominently for speech recognition in the 80s and 90s (Waibel et al., 1989; Bottou et al., 1990). ConvNets were subsequently applied to NLP tasks such as part-of-speech tagging and semantic role labelling (Collobert & Weston, 2008; Collobert et al., 2011; dos Santos & Zadrozny, 2014). More recently, convolutional networks were applied to sentence classification (Kalchbrenner et al., 2014; Kim, 2014) and document classification (Zhang et al., 2015; Conneau et al., 2017; Johnson & Zhang, 2015; 2017). Particularly inspiring for our work are the recent applications of convolutional architectures to machine translation (Kalchbrenner et al., 2016; Gehring et al., 2017a;b), audio synthesis (van den Oord et al., 2016), and language modeling (Dauphin et al., 2017).

1. CNN has also been used for sequences for a long time, and has recently achieved great results

- CNN은 예전부터 sequence task에서 사용되어 8-90년대 음성인식에 두드러졌고, 이후 NLP나 문장/문서 분류에도 활발히 사용되면서 최근에는 기계 번역, 오디오 합성 등에도 적용
- 즉, sequence는 RNN만의 영역이 아니다!

02 Background - 5 key points

Recurrent networks are dedicated sequence models that maintain a vector of hidden activations that are propagated through time (Elman, 1990; Werbos, 1990; Graves, 2012). This family of architectures has gained tremendous popularity due to prominent applications to language modeling (Sutskever et al., 2011; Graves, 2013; Hermans & Schrauwen, 2013) and machine translation (Sutskever et al., 2014; Bahdanau et al., 2015). The intuitive appeal of recurrent modeling is that the hidden state can act as a representation of everything that has been seen so far in the sequence. Basic RNN architectures are notoriously difficult to train (Bengio et al., 1994; Pascanu et al., 2013) and more elaborate architectures are commonly used instead, such as the LSTM (Hochreiter & Schmidhuber, 1997) and the GRU (Cho et al., 2014). Many other architectural innovations and training techniques for recurrent networks have been introduced and continue to be actively explored (El Hihi & Bengio, 1995; Schuster & Paliwal, 1997; Gers et al., 2002; Koutnik et al., 2014; Le et al., 2015; Ba et al., 2016; Wu et al., 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018).

2. RNNs are conceptually powerful, but are difficult to train, requiring elaborateness like LSTM/GRU

- 기본형 RNN은 gradient 소실/폭주로 인해 학습이 어려워, LSTM/GRU 같은 정교한 변형이 표준처럼 쓰이며, 이 외에도 수많은 변형 기법을 연구중

02 Background - 5 key points

Multiple empirical studies have been conducted to evaluate the effectiveness of different recurrent architectures. These studies have been motivated in part by the many degrees of freedom in the design of such architectures. Chung et al. (2014) compared different types of recurrent units (LSTM vs. GRU) on the task of polyphonic music modeling. Pascanu et al. (2014) explored different ways to construct deep RNNs and evaluated the performance of different architectures on polyphonic music modeling, character-level language modeling, and word-level language modeling. Jozefowicz et al. (2015) searched through more than ten thousand different RNN architectures and evaluated their performance on various tasks. They concluded that if there were “architectures much better than the LSTM”, then they were “not trivial to find”. Greff et al. (2017) benchmarked the performance of eight LSTM variants on speech recognition, handwriting recognition, and polyphonic music modeling. They also found that “none of the variants can improve upon the standard LSTM architecture significantly”. Zhang et al. (2016) systematically analyzed the connecting architectures of RNNs and evaluated different architectures on character-level language modeling and on synthetic stress tests. Melis et al. (2018) benchmarked LSTM-based architectures on word-level and character-level language modeling, and concluded that “LSTMs outperform the more recent models”.

3. The conclusion of the internal comparison of RNNs is that “LSTM is still the standard”

- 광범위한 벤치마크 하에서 LSTM vs GRU, RNN 구성 방식 등 수많은 연구 방식이 있었으나, LSTM보다 훨씬 나은 구조를 찾는 것은 쉽지 않았음
- 즉, 여전히 LSTM이 최신 모델들보다 성능이 좋음

02 Background - 5 key points

Other recent works have aimed to combine aspects of RNN and CNN architectures. This includes the Convolutional LSTM (Shi et al., 2015), which replaces the fully-connected layers in an LSTM with convolutional layers to allow for additional structure in the recurrent layers; the Quasi-RNN model (Bradbury et al., 2017) that interleaves convolutional layers with simple recurrent layers; and the dilated RNN (Chang et al., 2017), which adds dilations to recurrent architectures. While these combinations show promise in combining the desirable aspects of both types of architectures, our study here focuses on a comparison of generic convolutional and recurrent architectures.

4. There are also hybrid researches, but we focus on a head-on comparison of pure 'CNN vs RNN'

- RNN x CNN 하이브리드 혼합 시도도 있었지만 이 논문에서는 순수 CNN vs RNN의 정면 비교를 진행

02 Background - 5 key points

While there have been multiple thorough evaluations of RNN architectures on representative sequence modeling tasks, we are not aware of a similarly thorough comparison of convolutional and recurrent approaches to sequence modeling. (Yin et al. (2017) have reported a comparison of convolutional and recurrent networks for sentence-level and document-level classification tasks. In contrast, sequence modeling calls for architectures that can synthesize whole sequences, element by element.) Such comparison is particularly intriguing in light of the aforementioned recent success of convolutional architectures in this domain. Our work aims to compare generic convolutional and recurrent architectures on typical sequence modeling tasks that are commonly used to benchmark RNN variants themselves (Hermans & Schrauwen, 2013; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016).

5. The real gap in research in this area has been a comparison in 'sequence generation' perspective

- RNN 아키텍처들끼리의 평가는 많았으나, CNN vs RNN의 sequence modeling에 대한 동등 조건의 체계적 비교는 드물었다. (문장 및 문서 분류 수준에서의 비교는 있었지만, sequence 생성 과제는 요구 역량이 다름)

03 Temporal Convolutional Networks (TCN)

Designed “principle-centered” to achieve simplicity + autoregressive prediction + very long memory at the same time, referring to the ideas of recent CNNs for sequences (WaveNet, ConvS2S, Gated ConvNet)

TCN = (인과성을 지키는) 길이 보존 **1D FCN**

+ **Dilated Conv** (수용 영역 지수적 확대)

+ **Residual** (깊게 쌓아서 안정화)

- (WaveNet식 전층 skip/conditioning/gating/etc.)

→ 단순하면서도 매우 긴 memory를 갖는 general sequence!

03 Temporal Convolutional Networks (TCN)

3.1 Sequence Modeling

some corresponding outputs $y_0, \dots, y_T$ at each time. The key constraint is that to predict the output y_t for some time t , we are constrained to only use those inputs that have been previously observed: $x_0, \dots, x_t$. Formally, a sequence modeling network is any function $f : \mathcal{X}^{T+1} \rightarrow \mathcal{Y}^{T+1}$ that produces the mapping

$$\hat{y}_0, \dots, \hat{y}_T = f(x_0, \dots, x_T) \quad (1)$$

if it satisfies the causal constraint that y_t depends only on $x_0, \dots, x_t$ and not on any “future” inputs $x_{t+1}, \dots, x_T$. The goal of learning in the sequence modeling setting is to find a network f that minimizes some expected loss between the actual outputs and the predictions, $L(y_0, \dots, y_T, f(x_0, \dots, x_T))$, where the sequences and outputs are drawn according to some distribution.

- 입력 sequence : x / 각 시점마다 예측해야 할 출력이 y
- casual constraint: 시점 t 의 출력 y_t 는 오직 과거와 현재의 입력($x_0 \sim x_t$)에만 의존해야함 $\rightarrow$ leakage X
- encompasses auto-regressive prediction, but doesn't capture domains such as seq2seq (cuz this allows future inputs too)

Q. Then, how do we implement these mathematical definitions?

03 Temporal Convolutional Networks (TCN)

3.2 Casual Convolutions

<Two Principles>

A. The network produces an output of the same length as the input

B. There can be no leakage from the future into the past

A : TCN uses 1D FCN and to keep the length of all hidden layers equal to the input length, zero padding is added equal to (kernel size - 1)

→ 시점 별로 하나씩 출력하는 구조가 자연스럽게 생성됨

B : casual convolution (인과 합성곱) 사용 → 시점 t 의 출력이 바로 아래층의 t 이하 위치(과거/현재)와만 합성곱 되도록 kernel 배치

- BUT, 이런 “기본형”만으로 매우 긴 과거를 보려면 층을 엄청 깊게 하거나 커널을 크게 해야 한다는 단점....
- 즉 1D FCN만 층층이 쌓으면 과거를 보는 길이 (= 수용 영역, effective history)가 네트워크 깊이에 linear하게만 늘어나기에 긴 문맥이 필요한 경우 비효율적...

→ **dilation (팽창 합성곱)**을 도입해 깊이를 많이 늘리지 않아도 수용영역을 지수적으로 키우자!

To put it simply: $\text{TCN} = \text{1D FCN} + \text{causal convolutions.}$

03 Temporal Convolutional Networks (TCN)

3.3. Dilated Convolutions

history. Our solution here, following the work of van den Oord et al. (2016), is to employ dilated convolutions that enable an exponentially large receptive field (Yu & Koltun, 2016). More formally, for a 1-D sequence input $\mathbf{x} \in \mathbb{R}^n$ and a filter $f : \{0, \dots, k-1\} \rightarrow \mathbb{R}$, the dilated convolution operation F on element s of the sequence is defined as

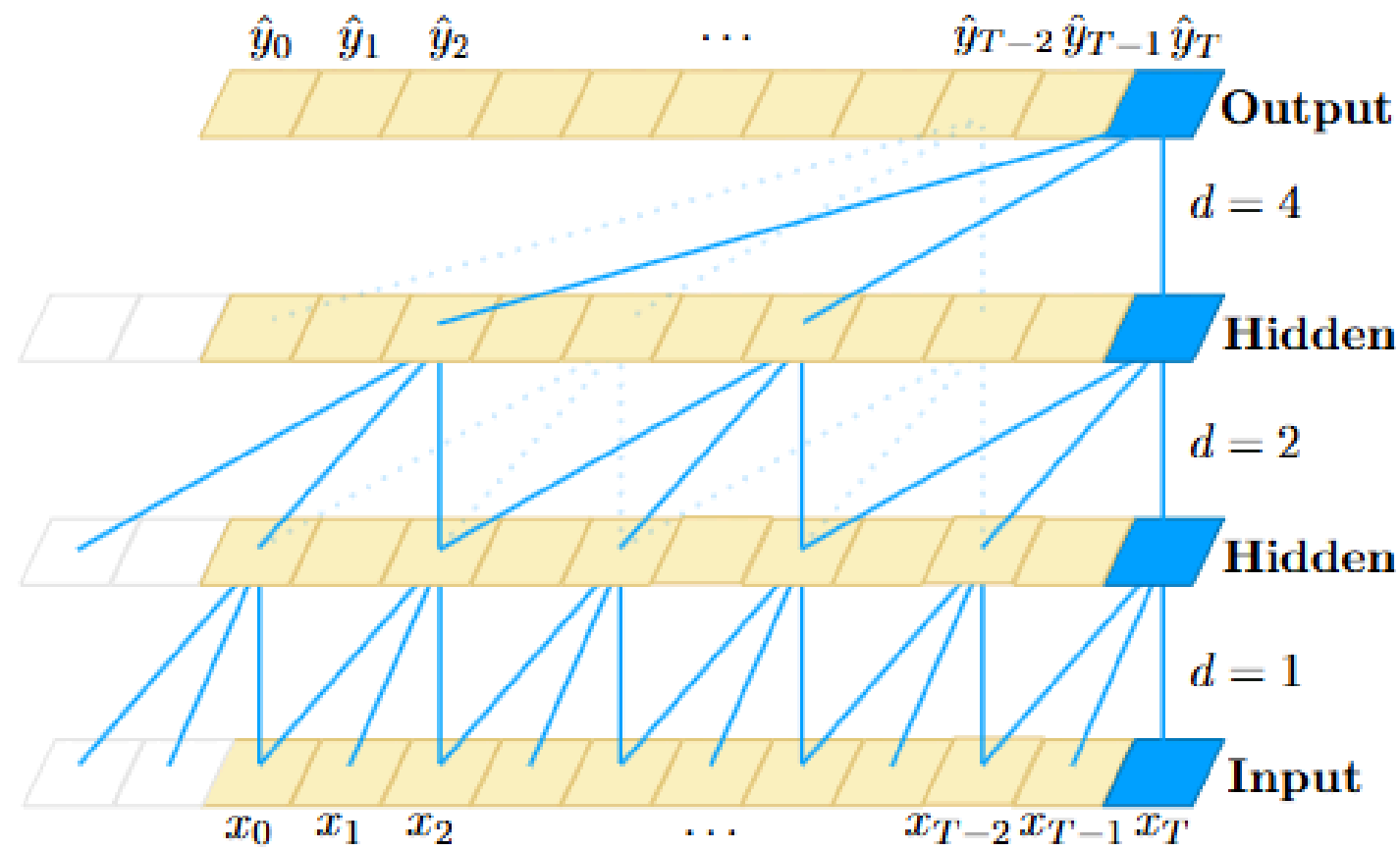
$$F(s) = (\mathbf{x} *_d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i} \quad (2)$$

- d : dilation factor / k : filter size
- $(k-1)d$: effective history of one layer
 - k or d 모두 키울 수 있지만, 보통은 층이 깊어질 수록 d 를 지수적으로 증가시키는 $d=O(2^i)$ 스케줄 사용

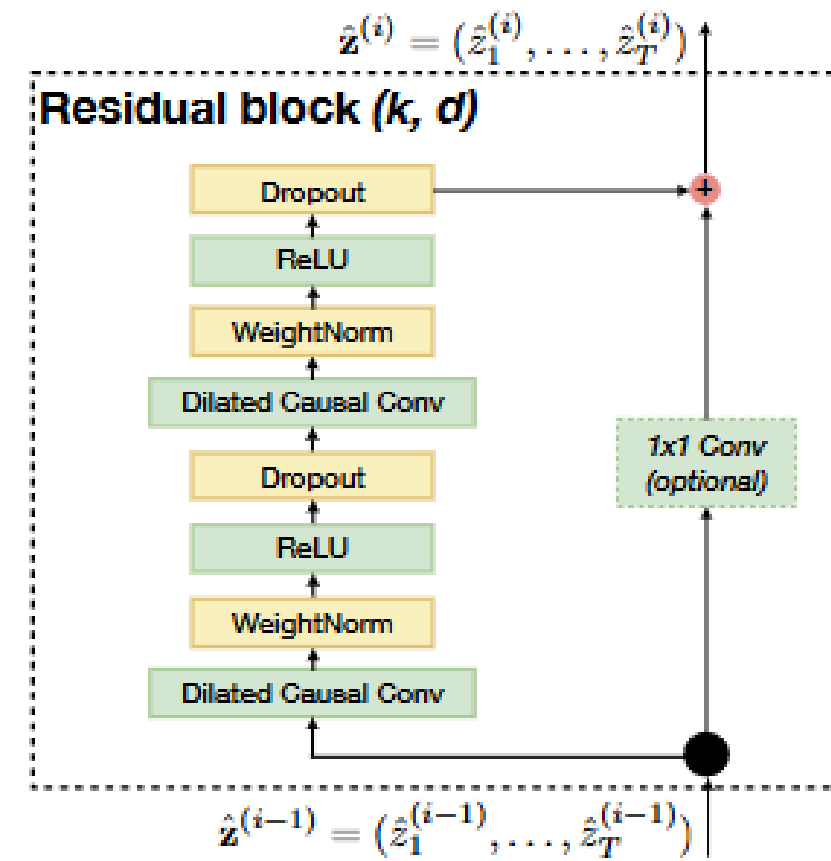
- $d=1 \rightarrow$ same as normal convolution
- $d>1 \rightarrow$ neighboring samples are skipped by “ d ” spaces, referencing further back in time
- Therefore, even with the same depth we can see a much wider range of the past

효과적으로 매우 긴 과거를 커버할 수 있고,
유효 과거 안의 모든 시점에 닿는 필터가 존재하게 됨!

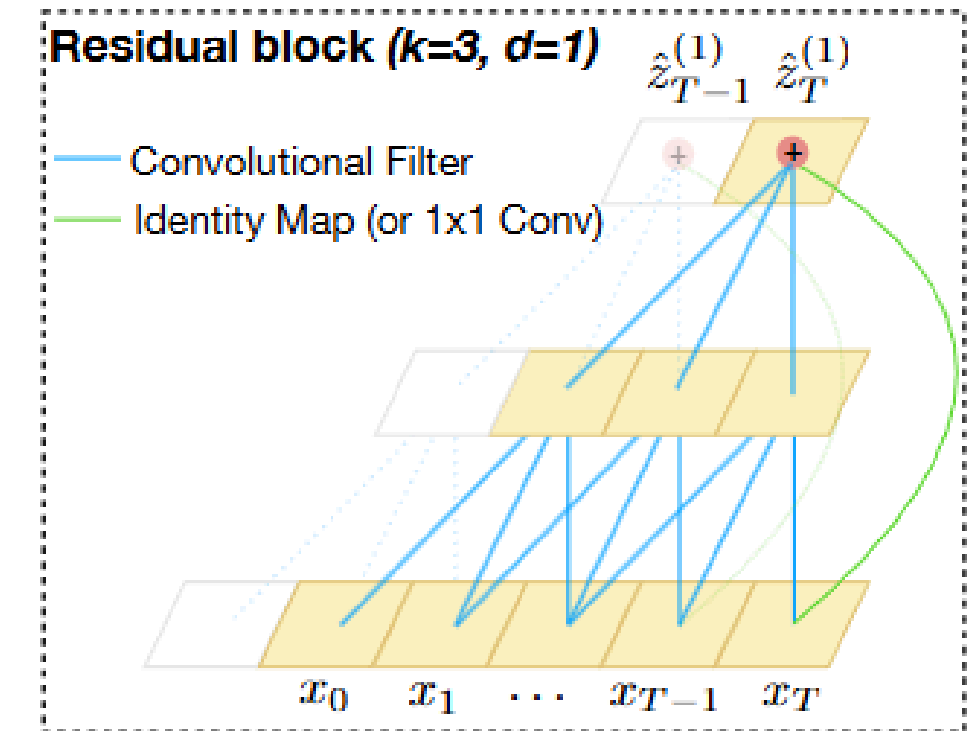
03 Temporal Convolutional Networks (TCN)



(a)



(b)



(c)

Figure 1. Architectural elements in a TCN. (a) A dilated causal convolution with dilation factors $d = 1, 2, 4$ and filter size $k = 3$. The receptive field is able to cover all values from the input sequence. (b) TCN residual block. An 1×1 convolution is added when residual input and output have different dimensions. (c) An example of residual connection in a TCN. The blue lines are filters in the residual function, and the green lines are identity mappings.

03 Temporal Convolutional Networks (TCN)

3.4. Residual Connections

- TCN의 수용영역은 깊이 n , 커널 k , 팽창 d 에 좌우됨
- 정말 긴 문맥(e.g., 2^{12})을 보려면 꽤 깊은 네트워크가 필요하고, 깊어질수록 학습 안정화가 중요
 - 합성곱층 대신 **Residual Block**을 기본 모듈로 사용하자

$$o = \text{Activation}(\mathbf{x} + \mathcal{F}(\mathbf{x})) \quad (3)$$

- 위 수식으로 계산하면 각 layer가 원본 $\mathbf{x}$ 에 대한 fine-tune(잔차)만 학습하면 되므로, 아주 깊은 네트워크에서도 학습이 잘 되는 것을 확인 가능
- 즉, Residual은 깊게 쌓기(긴 메모리 확보)를 가능하게 하고, 팽창 합성곱은 수용영역 지수 확대를 담당하여 둘을 합치면 단순 구조로도 아주 긴 문맥을 안정적으로 다룰 수 있음

03 Temporal Convolutional Networks (TCN)

3.5. Discussion - 5 Advantages

- **Parallelism.** Unlike in RNNs where the predictions for later timesteps must wait for their predecessors to complete, convolutions can be done in parallel since the same filter is used in each layer. Therefore, in both training and evaluation, a long input sequence can be processed as a whole in TCN, instead of sequentially as in RNN.
- **Flexible receptive field size.** A TCN can change its receptive field size in multiple ways. For instance, stacking more dilated (causal) convolutional layers, using larger dilation factors, or increasing the filter size are all viable options (with possibly different interpretations). TCNs thus afford better control of the model's memory size, and are easy to adapt to different domains.
- **Stable gradients.** Unlike recurrent architectures, TCN has a backpropagation path different from the temporal direction of the sequence. TCN thus avoids the problem of exploding/vanishing gradients, which is a major issue for RNNs (and which led to the development of LSTM, GRU, HF-RNN (Martens & Sutskever, 2011), etc.).
- **Low memory requirement for training.** Especially in the case of a long input sequence, LSTMs and GRUs can easily use up a lot of memory to store the partial results for their multiple cell gates. However, in a TCN the filters are shared across a layer, with the backpropagation path depending only on network depth. Therefore in practice, we found gated RNNs likely to use up to a multiplicative factor more memory than TCNs.
- **Variable length inputs.** Just like RNNs, which model inputs with variable lengths in a recurrent way, TCNs can also take in inputs of arbitrary lengths by sliding the 1D convolutional kernels. This means that TCNs can be adopted as drop-in replacements for RNNs for sequential data of arbitrary length.

03 Temporal Convolutional Networks (TCN)

3.5. Discussion - 2 Disadvantages

- **Data storage during evaluation.** In evaluation/testing, RNNs only need to maintain a hidden state and take in a current input x_t in order to generate a prediction. In other words, a “summary” of the entire history is provided by the fixed-length set of vectors h_t , and the actual observed sequence can be discarded. In contrast, TCNs need to take in the raw sequence up to the effective history length, thus possibly requiring more memory during evaluation.
 - **Potential parameter change for a transfer of domain.** Different domains can have different requirements on the amount of history the model needs in order to predict. Therefore, when transferring a model from a domain where only little memory is needed (i.e., small k and d) to a domain where much longer memory is required (i.e., much larger k and d), TCN may perform poorly for not having a sufficiently large receptive field.
- RNN = 과거 내용을 모두 요약한 hidden state 하나만 들고 다니는 방식
 - 학습할 때: 압축 과정의 중간 단계들을 다 저장해야 하기 때문에 무거움
 - 추론할 때: 압축본(h_t) 하나만 들고 다니면 되니 가벼움
 - TCN = 필요한 과거 구간을 통째로 들고 다니는 방식
 - 학습할 때: conv 구조 덕분에 메모리 공유가 잘 되어 가벼움
 - 추론할 때: 매번 conv를 하기 위해 원본인 data 버퍼를 유지해야 하므로 무거움

04 Sequence Modeling Tasks (1) Synthetic Tasks

- **Adding Problem**

- 입력: 길이 n 시퀀스, 대부분 0이고 임의의 두 위치만 표시됨
- 목표: 그 두 위치의 숫자를 더하기
- 핵심: 두 지점을 멀리 떨어뜨려 배치 → 장기 기억 필요
- 결과: $\text{TCN} \approx \text{GRU} \ll \text{LSTM}, \text{RNN}$ (TCN/GRU 압승)

- **Sequential MNIST / Permuted MNIST**

- 입력: MNIST 이미지를 784픽셀 순서대로 입력
- Permuted: 픽셀 순서를 무작위 섞음
- 핵심: 이미지 전체 문맥을 길게 기억해야 숫자를 인식 가능
- 결과: $\text{TCN} (97\sim99\%) \gg \text{LSTM/GRU} (85\sim96\%) \gg \text{RNN} (25\%)$

- **Copy Memory**

- 입력: 처음 10개의 숫자만 유효, 긴 공백 후 구분자 '9', 마지막에 그대로 복사 출력해야 함
- 핵심: 긴 공백 동안 초반 입력을 기억해야 하는 극단적 장기 의존성 문제
- 결과: $\text{TCN loss } 3.5\text{e-}5 \ll \text{RNN/LSTM/GRU (약 } 0.02) \rightarrow \text{TCN 압도적}$

=> Synthetic 테스트에서는 TCN이 장기 기억을 훨씬 더 잘 다룸

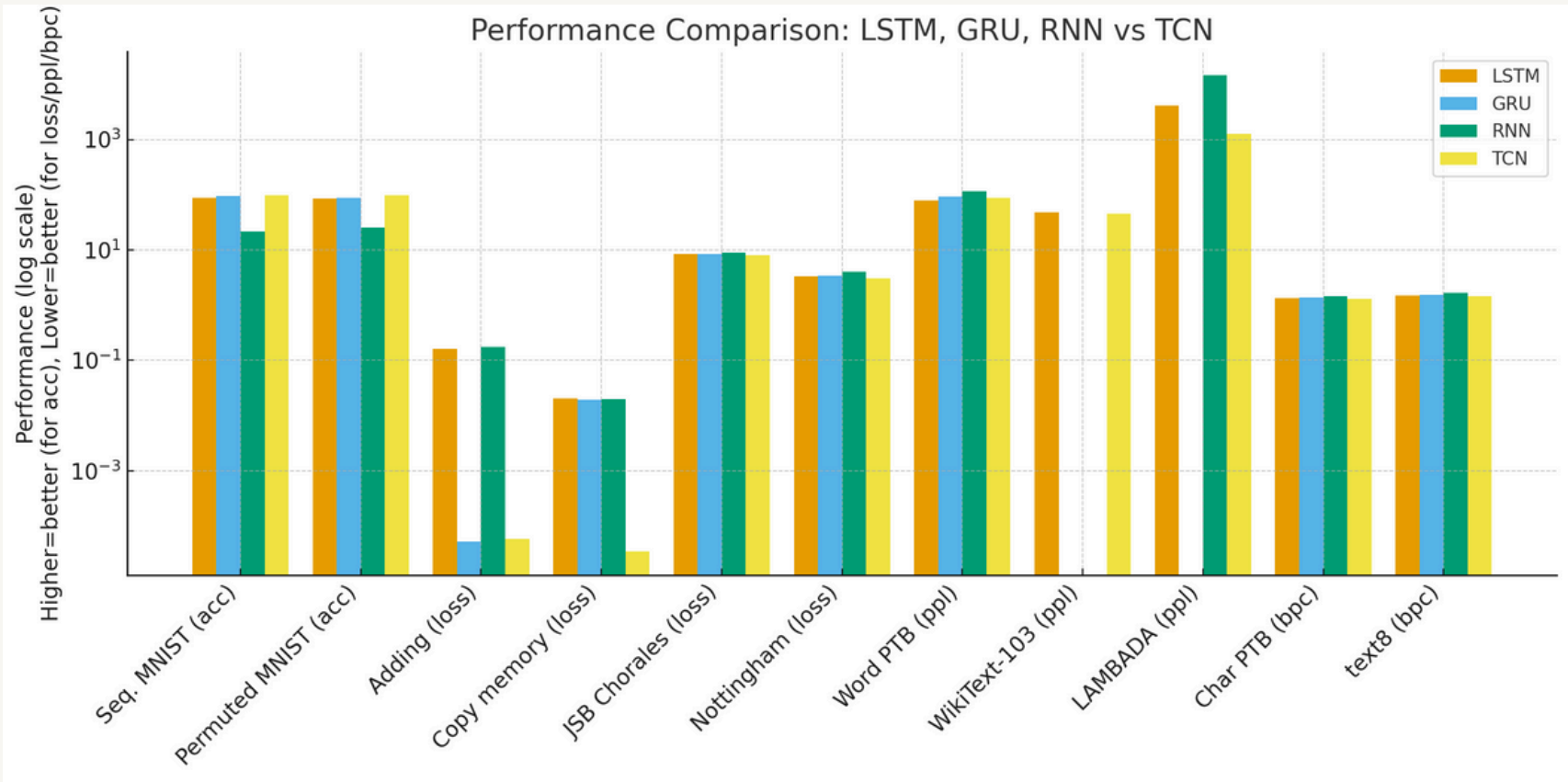
04 Sequence Modeling Tasks (2) Real-world Tasks

- **JSB Chorales & Nottingham (음악)**
 - 데이터: 합창곡, 민속곡 → 다음 음표 예측
 - 결과: TCN $\approx$ LSTM/GRU (비슷하거나 약간 우세)
- **Penn Treebank (PTB, word-level & char-level)**
 - Word-level: 작은 언어 데이터 → 예외적으로 LSTM이 더 나음 (78 vs TCN 88 perplexity)
 - Char-level: TCN이 더 낮은 BPC (1.31 vs LSTM 1.36)
- **WikiText-103**
 - 대규모, 현실적인 위키 데이터 → TCN이 더 낮은 perplexity (45 vs 48)
- **LAMBADA**
 - 긴 문맥에서 마지막 단어 맞추기 → 긴 의존성 필요
 - 결과: TCN 1279 $\ll$ LSTM 4186 $\ll$ RNN 14725 (압도적 성능)
- **text8 (char-level)**
 - 대규모 문자 기반 데이터 → TCN이 가장 낮은 BPC (1.45 vs LSTM 1.50)

=> 실제 데이터에서도 TCN이 대부분 RNN보다 우수
단, word-level PTB에서는 LSTM이 예외적으로 더 좋음

04 Sequence Modeling Tasks (3) Overall Results

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^h)	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^ℓ)	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^ℓ)	13M	78.93	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^ℓ)	3M	1.36	1.37	1.48	1.31
Char-level text8 (bpc)	5M	1.50	1.53	1.69	1.45



- Synthetic
 - TCN 압도적 (Adding, Copy Memory, SeqMNIST)
- Music
 - 비슷하지만 TCN 약간 우세
- Language
 - TCN이 대부분 우세 → 대규모 데이터 (LAMBADA, WikiText, text8)에서 특히 강력
- 예외: Word-level PTB → LSTM이 더 좋음

=> TCN은 거의 모든 벤치마크에서 LSTM/GRU보다 강력하며, 특히 긴 문맥·대규모 데이터에서 탁월한 성능을 보임

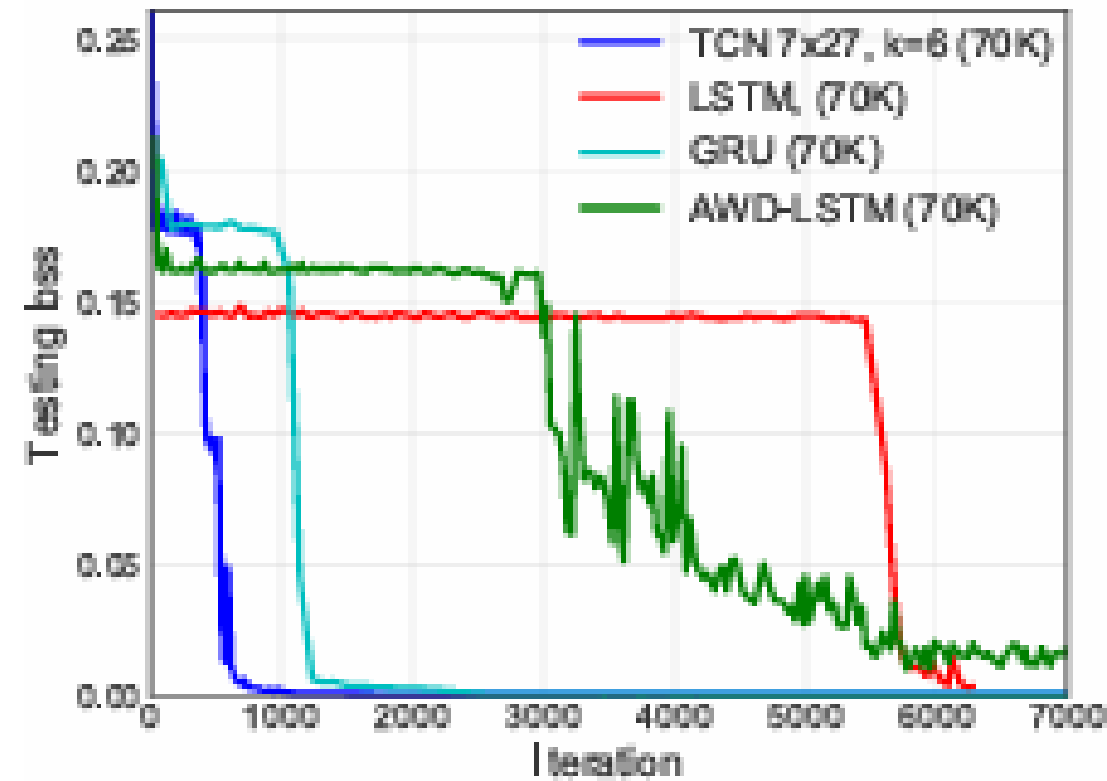
05 Experiments Setup

experiments reported in this section used exactly the same TCN architecture, just varying the depth of the network n and occasionally the kernel size k so that the receptive field covers enough context for predictions. We use an exponential dilation $d = 2^i$ for layer i in the network, and the Adam optimizer (Kingma & Ba, 2015) with learning rate 0.002 for TCN, unless otherwise noted. We also empirically find that gradient clipping helped convergence, and we pick the maximum norm for clipping from $[0.3, 1]$. When training recurrent models, we use grid search to find a good set of hyperparameters (in particular, optimizer, recurrent drop $p \in [0.05, 0.5]$, learning rate, gradient clipping, and initial forget-gate bias), while keeping the network around the same size as TCN. No other architectural elaborations, such as gating mechanisms or skip connections, were added to either TCNs or RNNs. Additional details and controlled experiments are provided in the supplementary material.

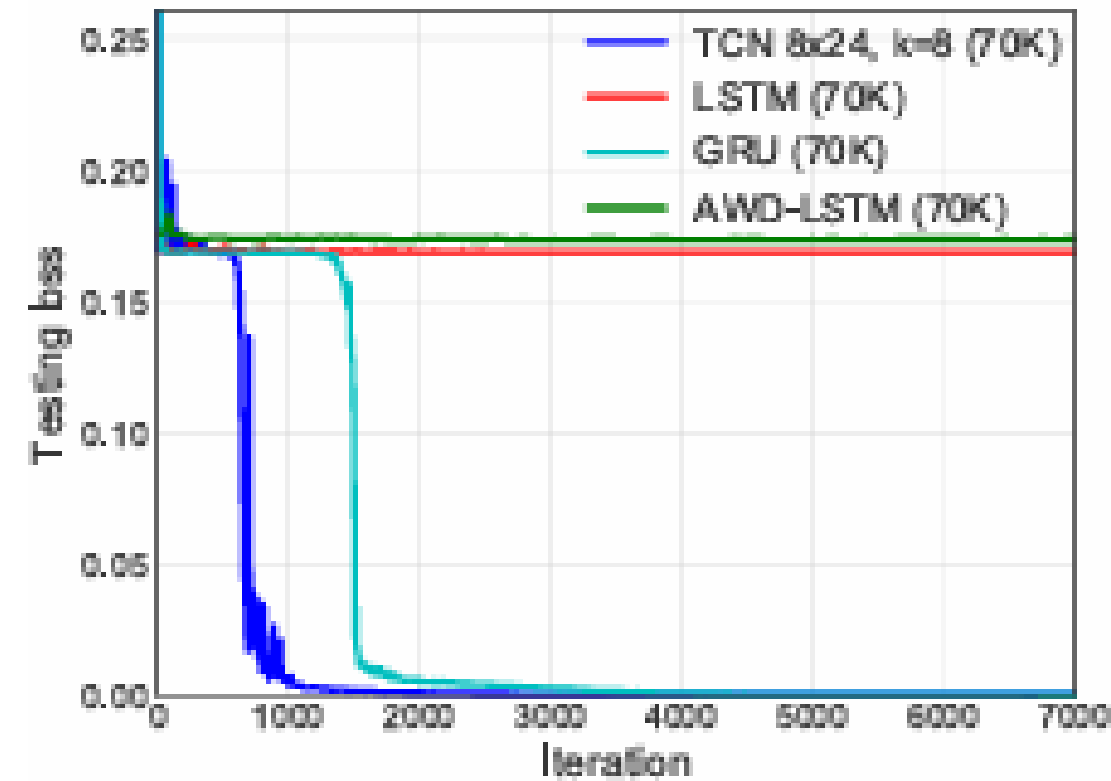
- 비교 대상: TCN vs LSTM, GRU, RNN (canonical recurrent architectures)
- 모델 크기: 비슷한 파라미터 수로 맞춤 (공정 비교)
- 구조 차이:
 - TCN: exponential dilation $d = 2^i$ 사용
 - RNN: grid search로 최적 hyperparameter 탐색
- 최적화 방식: Adam optimizer (learning rate = 0.002)
- Regularization:
 - gradient clipping $[0.3, 1]$
 - recurrent dropout ($p \in [0.05, 0.5]$)
- 공정성 확보: gating, skip connections 등 추가적 트릭은 사용하지 않음

=> 실험이 공정하게 세팅되었음

05 Experiments Synthetic Stress Tests (**Adding**, SeqMNIST, Copy Memory)



(a) $T = 200$

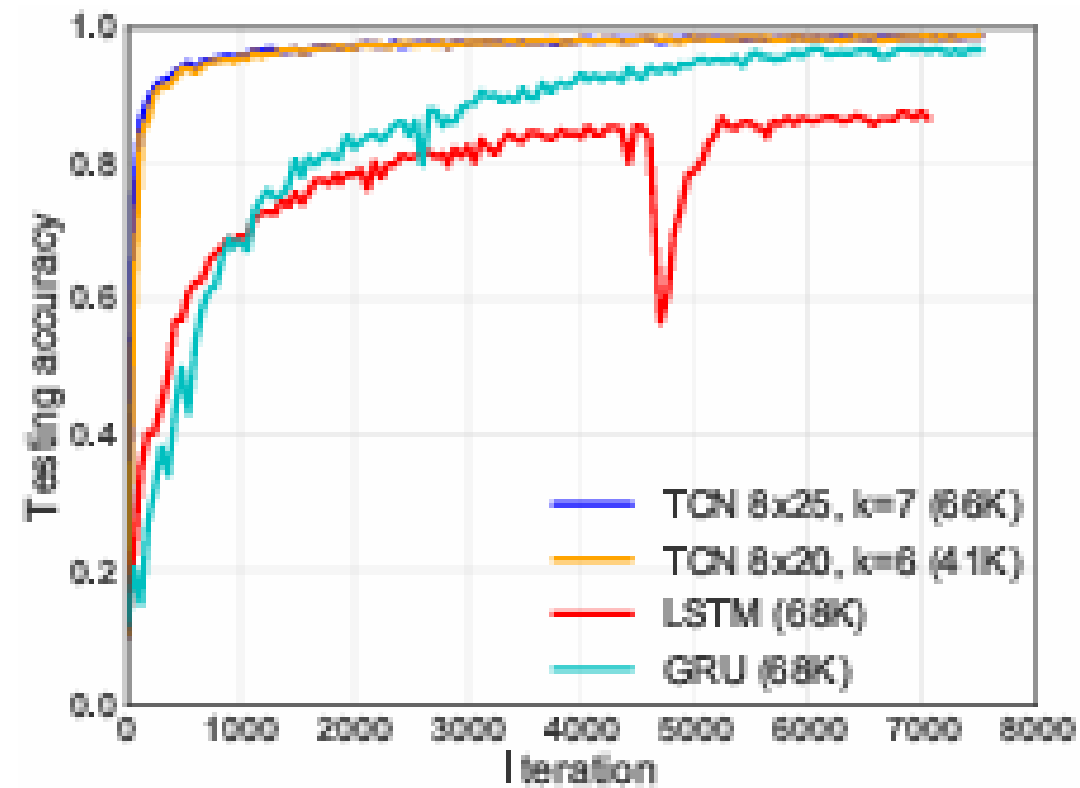


(b) $T = 600$

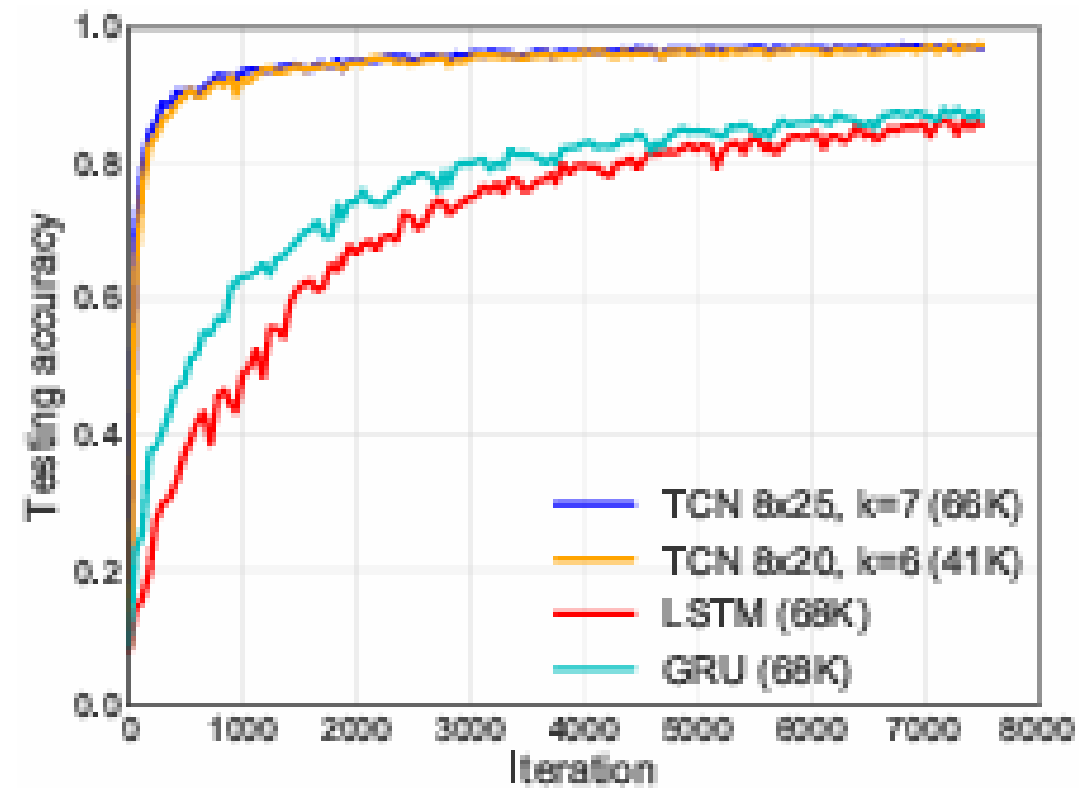
Figure 2. Results on the adding problem for different sequence lengths T . TCNs outperform recurrent architectures.

- Adding Problem
 - 랜덤 시퀀스에서 두 값만 골라 합해야 함 → 단순하지만 장기 기억 필요
 - 결과: TCN은 빠르게 $MSE \approx 0$ 에 수렴, GRU도 준수, LSTM/RNN은 저조

05 Experiments Synthetic Stress Tests (Adding, SeqMNIST, Copy Memory)



(a) Sequential MNIST



(b) P-MNIST

Figure 3. Results on Sequential MNIST and P-MNIST. TCNs outperform recurrent architectures.

- Sequential MNIST / Permuted MNIST
 - 이미지 픽셀을 순서대로 입력 (또는 순서 섞기)
 - 결과: TCN 정확도 97~99% (SOTA 수준), LSTM/GRU는 85~96%, RNN은 25% 근처
 - TCN은 수렴 속도·최종 정확도 모두 압도적

05 Experiments Synthetic Stress Tests (Adding, SeqMNIST, **Copy Memory**)

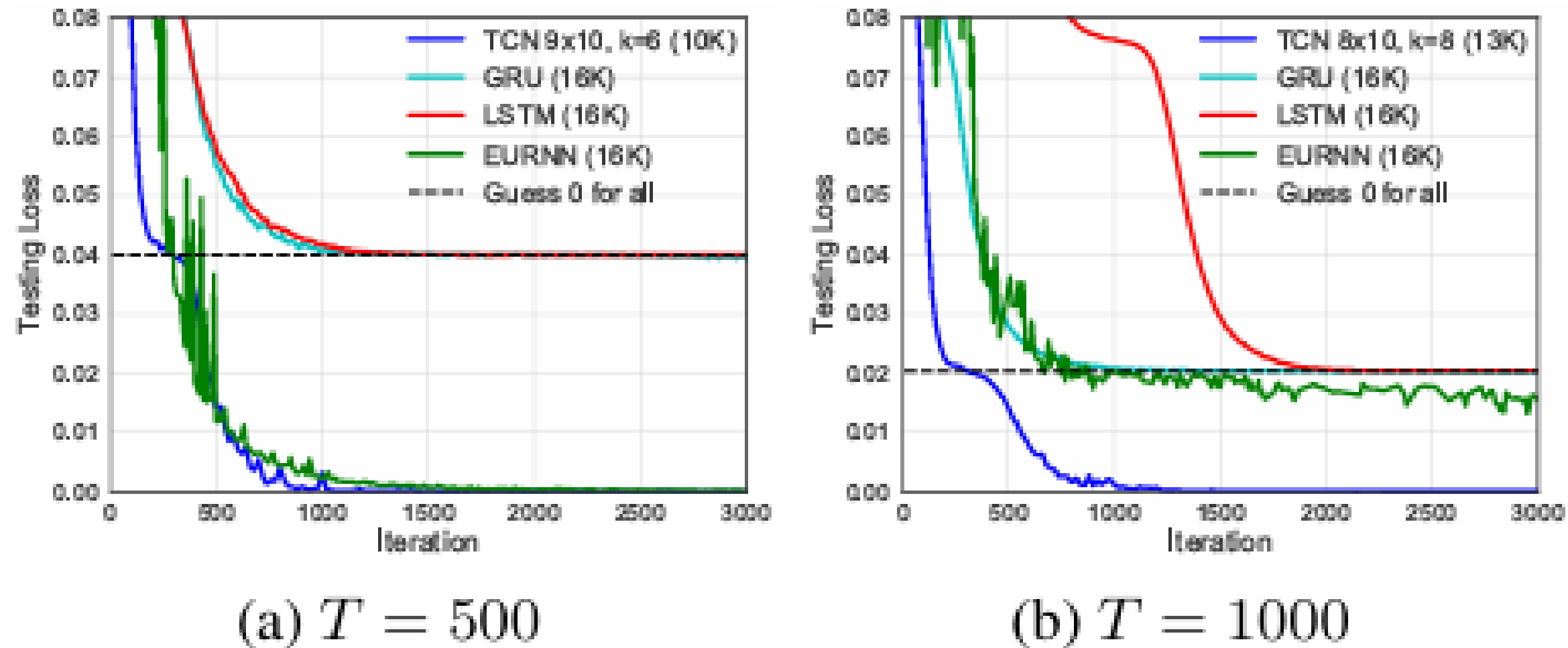
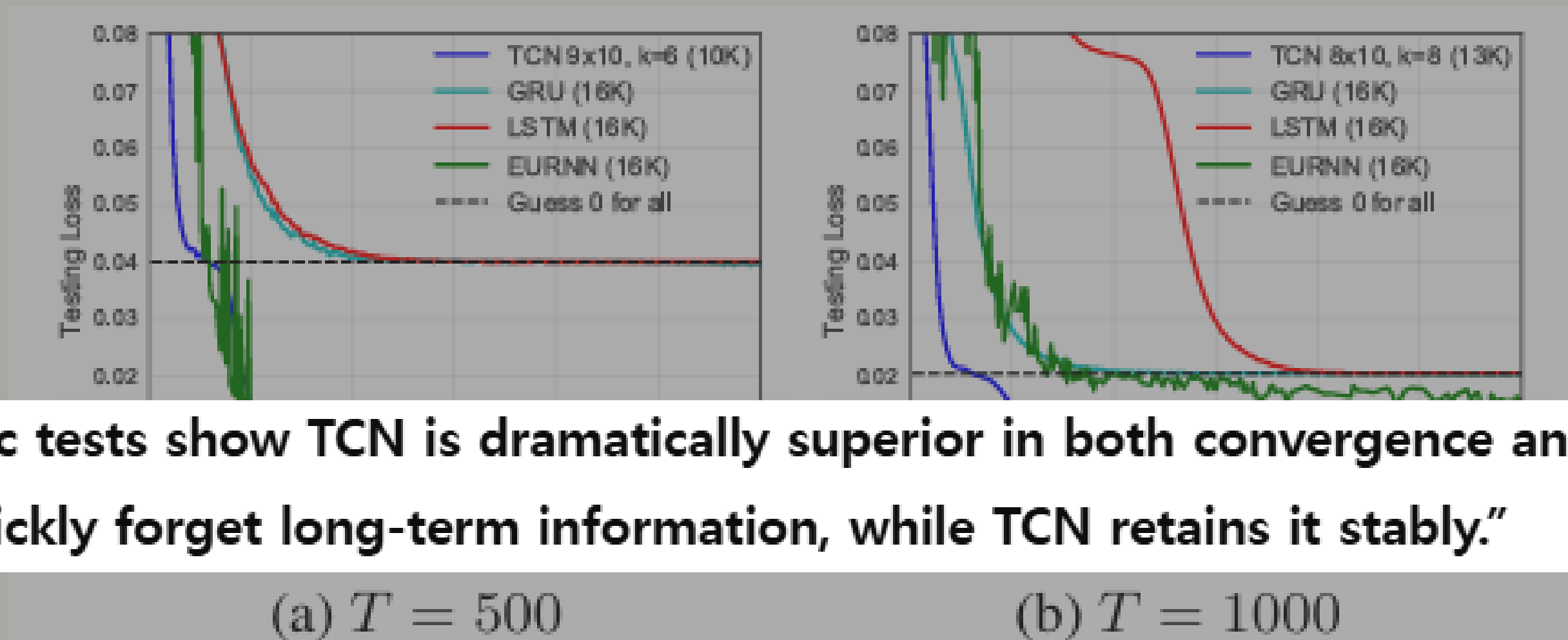


Figure 4. Result on the copy memory task for different sequence lengths T . TCNs outperform recurrent architectures.

- Copy Memory
 - 처음 10개 입력을 긴 공백 뒤 복사해야 함 (극단적 장기 의존성)
 - 결과: TCN은 정답에 빠르게 수렴, LSTM/GRU는 대부분 "0"만 출력
 - $T=1000$ 에서도 TCN은 안정적, RNN류는 붕괴

05 Experiments Synthetic Stress Tests (Adding, SeqMNIST, **Copy Memory**)



“Synthetic tests show TCN is dramatically superior in both convergence and accuracy. RNNs quickly forget long-term information, while TCN retains it stably.”

Figure 4. Result on the copy memory task for different sequence lengths T . TCNs outperform recurrent architectures.

- Copy Memory
 - 처음 10개 입력을 긴 공백 뒤 복사해야 함 (극단적 장기 의존성)
 - 결과: TCN은 정답에 빠르게 수렴, LSTM/GRU는 대부분 “0”만 출력
 - $T=1000$ 에서도 TCN은 안정적, RNN류는 붕괴

05 Experiments

Real-world Tasks: Music & Language Modeling

- **Polyphonic Music (JSB Chorales, Nottingham)**
 - 음표 시퀀스를 예측하는 과제
 - 결과: TCN이 RNN 계열보다 consistently 낮은 loss
 - 일부 특수 RNN 모델(HF-RNN 등)은 더 강력했지만, 기본 구조만 비교 시 TCN 우세
- **Word-level Language Modeling**
 - Penn Treebank (PTB): 작은 데이터셋 → 최적화된 LSTM이 TCN보다 낮은 perplexity (예외 케이스)
 - WikiText-103, LAMBADA: 대규모 데이터 → TCN이 훨씬 낮은 perplexity
 - 특히 LAMBADA는 긴 문맥 이해가 필요한 과제 → TCN 압도적
- **Character-level Language Modeling (PTB, text8)**
 - 단어가 아닌 문자 단위 예측
 - 결과: TCN이 LSTM/GRU보다 꾸준히 낮은 BPC

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^h)	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^ℓ)	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^ℓ)	13M	78.93	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^ℓ)	3M	1.36	1.37	1.48	1.31
Char-level text8 (bpc)	5M	1.50	1.53	1.69	1.45

=> “실제 과제에서도 대부분 TCN이 RNN을 능가하며,
단 Word-level PTB만 LSTM이 예외적으로 우세하다.”

05 Experiments Memory Size: TCN vs RNNs

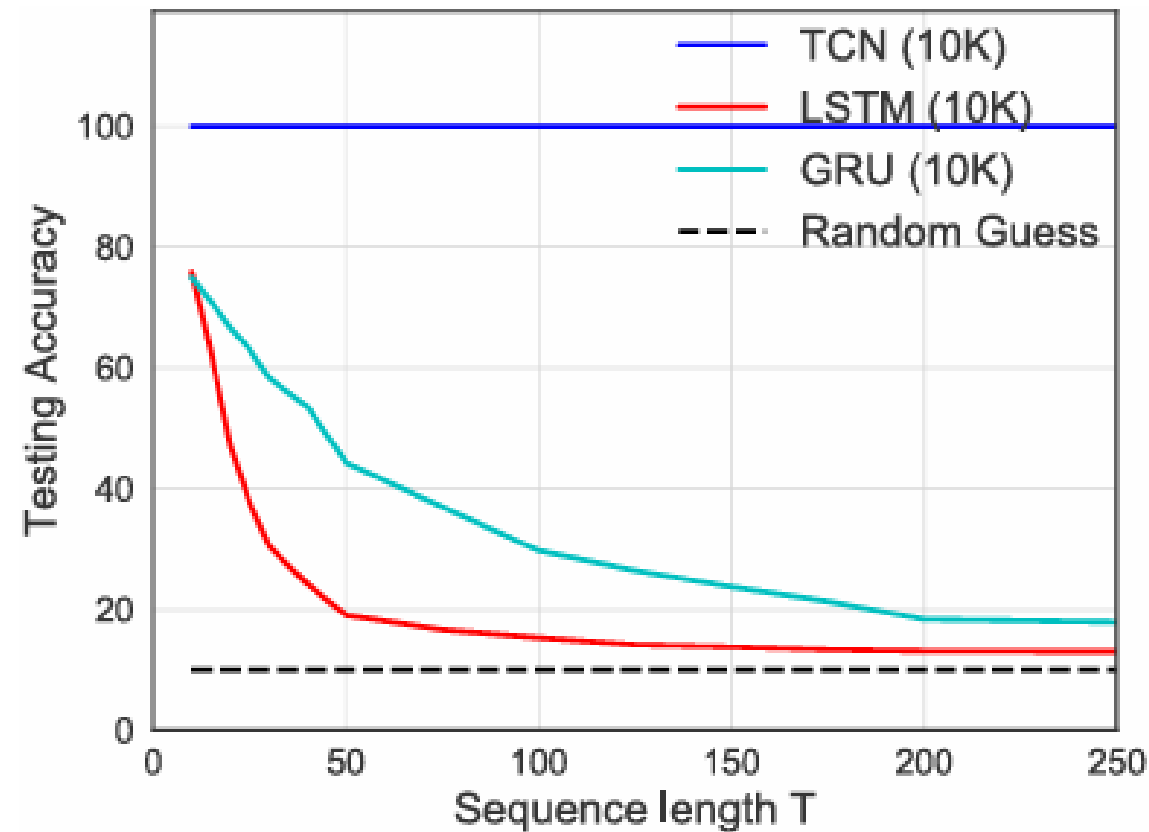


Figure 5. Accuracy on the copy memory task for sequences of different lengths T . While TCN exhibits 100% accuracy for all sequence lengths, the LSTM and GRU degenerate to random guessing as T grows.

- 그래프에서 TCN은 안정적으로 높은 accuracy 유지,
- LSTM/GRU는 길이가 늘어날수록 급격히 성능 저하.

- 이론적 전제
 - RNN (특히 LSTM/GRU): 무한 장기 기억 가능하다고 설계됨
 - 하지만 실제 학습에서는 gradient 소실 때문에 긴 시퀀스를 기억 못 함
- 실험 결과
 - Copy Memory (길이별):
 - TCN: 길이가 1000 이상이어도 정확도 유지
 - LSTM: $T < 50$ 부터 성능 급락
 - GRU: $T < 200$ 부터 급락
 - LAMBADA: 긴 문맥 이해 필요 → TCN이 LSTM/GRU보다 훨씬 낮은 perplexity
- 핵심 의미
 - TCN은 실제로 장기 의존성(long-term dependency)을 훨씬 잘 다룸
 - RNN의 “이론적 장점”이 실제에서는 발휘되지 않음

=> “TCN은 실제로도 RNN보다 훨씬 긴 기억을 안정적으로 유지한다.”

06 Conclusion

- 연구 요약
 - TCN vs RNN (LSTM, GRU) 비교 → 다양한 시퀀스 과제에서 평가
- 주요 결과
 - TCN이 대부분의 과제에서 RNN보다 우수
 - RNN의 “무한 기억” 장점은 실제로는 거의 발휘되지 않음
 - 같은 규모일 때 TCN이 더 긴 메모리 유지
- LSTM vs TCN
 - LSTM은 커뮤니티 최적화 덕분에 발전
 - TCN은 아직 최적화 연구가 부족 → 발전 가능성 큼
- 시사점
 - RNN이 시퀀스 모델링의 표준이던 시대는 끝나가고 있음
 - TCN은 단순·명료하면서도 강력 → 시퀀스 모델링의 새로운 출발점

=> “TCN은 단순하고 강력하며, 시퀀스 모델링의 새로운 출발점이 될 수 있다.”

The End.
